TUTORIAL 2 OF 4 — DETAILED EDITION

Multi-MCU and FreeRTOS

UART Protocol · Task Architecture · IPC · Watchdog · OTA Atomicity · Safety

UART Protocol	FreeRTOS Tasks	IPC Patterns	Deadlock/Stack	OTA Atomicity

Every concept explained from first principles, with working code and the reasoning behind every decision.

1. Why Three MCUs — The Architectural Reasoning

Before studying the inter-MCU protocol, you need to understand why the 3-MCU architecture exists. It is not arbitrary complexity — it reflects a deliberate safety and certification strategy.

A single MCU running everything has one critical weakness: a firmware bug anywhere in the system can affect everything, including the safety functions. If the WiFi stack has a memory corruption bug (which happens — it is complex software), it could corrupt the control algorithm's stack and cause the actuator to behave incorrectly. This is a Class C risk scenario.

Separating the MCUs creates **fault isolation boundaries**. The Sensor MCU knows nothing about WiFi, MQTT, or OTA — it cannot be affected by a bug in the network stack. The Control MCU runs the closed-loop algorithm on its own isolated processor with its own watchdog. The Comms MCU handles all cloud communication and can crash and restart without interrupting the control loop.

From a regulatory perspective, separating safety functions onto dedicated hardware also reduces the software safety class of each individual MCU. The Control MCU — which implements the closed-loop algorithm — is Class B or C. The Comms MCU — which handles networking but does not directly control actuators — may be arguable as Class B even though the overall system is higher risk. This simplifies your IEC 62304 compliance burden.

> *Think of the 3-MCU system like a hospital. The ICU (Control MCU) handles critical patients and has its own dedicated staff. The communications department (Comms MCU) handles phone calls and information systems. If the phone system crashes, the ICU keeps running. The two departments communicate through defined channels with defined protocols — not by sharing the same staff.*

2. UART Frame Protocol — Why Every Field Exists

A UART frame protocol seems simple on the surface — just put some bytes in a specific order. But every field in the frame format exists to solve a specific, real production problem. Understanding the reasoning will help you design better protocols and debug problems faster.

Why SOF and EOF Bytes?

The Start of Frame (SOF = 0xAA) and End of Frame (EOF = 0x55) bytes serve as synchronization markers. UART has no concept of frame boundaries — bytes arrive in a continuous stream. Without synchronization markers, if your receiver loses track of its position in the stream (due to a reset, a missed byte, or electrical noise), it has no way to find the next valid frame.

When the receiver detects 0xAA, it knows a new frame is starting. When it expects 0x55 at the end but sees something else, it knows the frame is invalid and discards it. This is why the state machine resets to IDLE when the EOF check fails — it discards the corrupted frame and immediately starts looking for the next SOF.

You might ask: what if 0xAA appears in the payload data? This is why the CRC is also checked. A byte that happens to be 0xAA mid-payload will not trigger a false SOF detection because the receiver is in PAYLOAD state, not IDLE state. The state machine's context disambiguates the SOF value.

Why a 16-bit Length Field?

The two-byte length field (LEN_H and LEN_L) allows payloads up to 65,535 bytes. For most inter-MCU messages (a 20-byte sensor reading, a 4-byte setpoint), this is far more than needed. The reason for the 16-bit field is future-proofing and OTA: when transferring firmware chunks over UART, chunks up to 512 bytes significantly improve throughput compared to smaller chunks, because each chunk has fixed overhead (SOF, CMD, LEN, CRC, EOF = 7 bytes) and you want to minimize the overhead-to-data ratio.

Why CRC-16 Instead of a Simple Checksum?

A simple arithmetic checksum (sum all bytes, take the low 8 bits) can detect single-byte errors, but it has a dangerous weakness: if two bytes in the payload are corrupted in complementary ways (one increases by X, another decreases by X), the checksum is unchanged and the corruption is invisible.

CRC-16 uses polynomial division to create a 16-bit check value that is sensitive to **burst errors** — consecutive bits flipped by EMI, cable noise, or ground bounce. The CRC-16/CCITT-FALSE polynomial (0x1021) can detect all single-bit errors, all double-bit errors, all burst errors of length 16 or less, and most longer burst errors. For a medical device running near motors, pumps, or other EMI sources, this error detection capability is not optional.

```
// Why CRC16/CCITT-FALSE and not CRC32? // CRC32 provides better error detection for large payloads, // but adds 2 extra bytes per frame. For short medical IoT frames // (typically <50 bytes), CRC16 is the right tradeoff. // CRC32 is appropriate when frames exceed ~512 bytes. // The CRC covers CMD + LEN_H + LEN_L + PAYLOAD. // It does NOT cover SOF or EOF -- these are synchronization bytes, // not data, and their corruption is detected by the state machine. uint16_t crc16_update(uint16_t crc, uint8_t byte) { crc ^= ((uint16_t)byte << 8); // XOR byte into high bits for (int i = 0; i < 8; i++) { // process each bit if (crc & 0x8000) // if high bit set: crc = (crc << 1) ^ 0x1021; // shift left, XOR poly else crc = (crc << 1); // just shift left } return crc; } // Known test: crc16_buf("123456789", 9) must equal 0x29B1 // Always verify your implementation against this vector.
```

The 50ms Inter-Byte Timeout — Why It Is Necessary

Without a timeout, a partial frame can lock your RX state machine forever. Imagine the sender is halfway through a frame and the UART cable is briefly disconnected or the sender reboots. The receiver has received SOF, CMD, LEN — it is in the PAYLOAD state waiting for more bytes. Without a timeout, it stays there indefinitely, ignoring all new frames.

The 50ms timeout says: if no new byte arrives within 50ms while a frame is in progress, this frame is dead — reset to IDLE. At 115200 baud, a byte takes about 87 microseconds. 50ms is approximately 575 byte-times, which is more than enough for any valid frame but short enough to recover quickly after a sender failure.

3. FreeRTOS — How Preemption Actually Works

Many engineers understand FreeRTOS task priorities conceptually but not mechanically. This gap causes subtle bugs. Understanding exactly when and how FreeRTOS switches between tasks will make you a much better embedded firmware engineer.

The Scheduler — Tick-Based Preemption

FreeRTOS has a system tick — a hardware timer interrupt that fires at a configured rate (typically 1000 Hz, giving a 1ms tick period). On every tick, the scheduler checks whether the currently running task should continue or be preempted.

The scheduler always runs the **highest-priority ready task**. 'Ready' means not blocked, not suspended, not waiting for a queue, semaphore, or delay. If a higher-priority task becomes ready at any moment — including mid-instruction — it will preempt the current task at the next scheduler opportunity. With configUSE_PREEMPTION=1, this happens at every tick.

This is why task priorities matter so much for safety. If your watchdog task is at the same priority as your sensor task, a busy sensor task can prevent the watchdog from running. The watchdog then fires a false alarm — or worse, a real hang goes undetected because the watchdog is starved. The watchdog must always be at a higher priority than any task it monitors.

Blocking vs Non-Blocking IPC — The Critical Distinction

Every FreeRTOS IPC function has a timeout parameter. This timeout determines what happens if the operation cannot complete immediately. This is one of the most important design decisions in your task architecture.

In a sensor task running at 100 Hz, you call xQueueSend to pass a reading to the control task. What if the queue is full because the control task is slow? If you use portMAX_DELAY (block forever), your sensor task will block indefinitely, missing its 10ms deadline and preventing the watchdog from being kicked. If you use a timeout of 0 (non-blocking), the send fails silently and you drop a reading.

The right approach depends on the semantics. For sensor data, a timeout of 0 is often correct — drop the reading and log it, because the system must keep running. For a setpoint command that must not be lost, block with a short timeout and escalate to a fault if the timeout expires, because a missed command could cause a safety problem.

```
// Design decision documented explicitly: void sensor_task(void *pv) { TickType_t xLast =
xTaskGetTickCount(); for (;;) { sensor_data_t d; sensor_read(&d); // Non-blocking send: if queue is full,
drop reading and log. // Rationale: control task overloaded is a fault condition; // we want to detect it,
not silently block and miss our deadline. if (xQueueSend(xControlQ, &d, 0) != pdTRUE) {
g_dropped_readings++; // observable metric if (g_dropped_readings > 10)
fault_escalate(FAULT_QUEUE_OVERFLOW); } watchdog_kick(WDT_BIT_SENSOR); // vTaskDelayUntil: delays until
absolute time, not relative. // This compensates for the time spent in sensor_read(). // Result: accurate
100Hz period regardless of execution time. vTaskDelayUntil(&xLast, pdMS_TO_TICKS(10)); } }
```

Why vTaskDelayUntil Instead of vTaskDelay

vTaskDelay(10ms) means 'wait 10ms from now'. If your task took 2ms to do its work, you wake up 12ms after the last wake-up — your period drifts over time. For a 100 Hz sensor task, this drift accumulates and your effective sample rate decreases.

vTaskDelayUntil(&xLastWake;, 10ms) means 'wake up at the next 10ms boundary from the last wake-up time'. It compensates for execution time automatically. If your task took 2ms, it waits only 8ms. The period remains exactly 10ms. For any periodic task — especially sensor acquisition or control loops — always use vTaskDelayUntil.

4. Choosing the Right IPC — A Decision Framework

The four main FreeRTOS IPC mechanisms — queues, semaphores, mutexes, and event groups — are frequently misused. Using the wrong one creates either correctness bugs (lost data, missed events) or safety bugs (priority inversion, deadlock). The choice is not about preference; it follows directly from what you are trying to accomplish.

Queue: For Transferring Data

Use a queue when you need to transfer actual data values between tasks. Queues are FIFO buffers — they preserve the order of items. Each item in a queue is a copy of your data structure. If the queue is full (sender faster than receiver), the sender can block, return an error, or overwrite the oldest item depending on the API you use.

The queue depth you choose represents how many items can be pending. A depth of 1 means 'only one unprocessed reading at a time — if the receiver hasn't processed the last one, the sender must wait or drop'. A depth of 10 means the sender can get 10 readings ahead of the receiver before blocking. Choose the depth based on how much latency is acceptable in your system.

Semaphore: For Signaling Events

Use a semaphore when you need to signal that something happened, without transferring data. A binary semaphore is a one-bit signal: 'event occurred'. A counting semaphore counts occurrences: 'N events occurred, process them'. The key difference from a queue: a semaphore carries no data — just the fact that something happened.

A common use case: an ISR detects a hardware event (DMA transfer complete, external interrupt) and needs to wake a task to process it. The ISR calls xSemaphoreGiveFromISR. The task is blocked on xSemaphoreTake. The moment the ISR gives the semaphore, the scheduler wakes the task. No data is transferred in the semaphore — the task reads the data directly from the hardware register or DMA buffer.

Mutex: For Protecting Shared Resources — NOT for Signaling

A mutex (mutual exclusion semaphore) is fundamentally different from a binary semaphore despite looking similar in the API. The key difference is **ownership and priority inheritance**. A mutex can only be given back by the task that took it. More importantly, FreeRTOS automatically raises the priority of a low-priority task holding a mutex to the priority of any higher-priority task waiting for it.

This priority inheritance is what prevents priority inversion — the scenario where a high-priority task is indefinitely starved because a low-priority task holds a resource it needs. If you use a binary semaphore for mutual exclusion (a common mistake), there is no priority inheritance and priority inversion can occur. The Mars Pathfinder mission in 1997 experienced a real-world priority inversion bug caused by exactly this mistake — the spacecraft reset repeatedly and nearly lost contact with Earth.

```
// WRONG: binary semaphore for mutual exclusion // No priority inheritance -> priority inversion possible
SemaphoreHandle_t xSemaphoreCreateBinary(); // ← do NOT use for mutex // RIGHT: proper mutex with priority
inheritance // configUSE_MUTEXES must be 1 in FreeRTOSConfig.h SemaphoreHandle_t xSharedBufMutex =
xSemaphoreCreateMutex(); // Low-priority task L holds the mutex: xSemaphoreTake(xSharedBufMutex,
portMAX_DELAY); // ... access shared buffer ... // If High-priority task H tries to take mutex now: //
FreeRTOS RAISES task L's priority to match task H's priority // Medium-priority task M can no longer preempt
task L // Task L finishes, gives mutex, H gets it immediately // Task L's priority returns to normal
xSemaphoreGive(xSharedBufMutex);
```

5. Priority Inversion — What It Is and Why It Kills Systems

Priority inversion is one of the most dangerous concurrency bugs because it manifests intermittently, is hard to reproduce, and can cause real-time deadlines to be missed in ways that look like random system instability.

The scenario: you have three tasks — Low (L), Medium (M), and High (H), in ascending priority order. L acquires a binary semaphore to protect shared data. While L holds the semaphore, H becomes ready and tries to acquire the same semaphore. H blocks because L holds it. Now M becomes ready. Since M has higher priority than L, M preempts L. L cannot run, so it cannot release the semaphore. H remains blocked indefinitely, waiting for a semaphore held by L, which cannot run because M keeps preempting it.

The effective priority of H — the highest-priority task — has been inverted to below M. If H has a hard real-time deadline (like a safety watchdog check), that deadline will be missed. The system appears to hang or behave erratically with no obvious cause in the logs.

The fix is a mutex with priority inheritance. When H tries to take the mutex held by L, FreeRTOS temporarily elevates L's priority to match H's. Now L has higher priority than M. M cannot preempt L. L completes its critical section, releases the mutex, and H immediately takes it. L's priority returns to its normal level. The inversion is prevented.

KEY INSIGHT: Always use xSemaphoreCreateMutex() (not binary semaphore) whenever you need mutual exclusion in a multi-priority task environment. This single rule prevents priority inversion and has no performance downside in most embedded systems.

6. Deadlock — How It Happens and How to Prevent It

Deadlock is a state where two or more tasks are each waiting for a resource held by the other. Neither can proceed. The system freezes silently — tasks appear to be running (the scheduler is still ticking) but no work gets done.

The classic scenario: Task A acquires Mutex X, then tries to acquire Mutex Y. Task B acquires Mutex Y, then tries to acquire Mutex X. If A acquires X and B acquires Y, then A blocks waiting for Y (held by B) and B blocks waiting for X (held by A). Both are permanently blocked.

The solution is a **global lock ordering rule**: all mutexes in the system are numbered, and every task must always acquire them in ascending order. If Mutex X is number 1 and Mutex Y is number 2, the rule says always acquire X before Y. Task A follows this rule (X then Y). Task B must also follow this rule (X then Y, not Y then X). Now there is no circular dependency.

This rule must be documented in your architecture spec and enforced in code reviews. It is also worth adding to your automated test suite: a deadlock detector that attempts to acquire all mutexes in every order and verifies no cycle is possible.

```
// Document your mutex ordering in the architecture spec: // Mutex acquisition order (always acquire in this sequence): // 1. g_sensor_mutex (protects sensor calibration data) // 2. g_control_mutex (protects control loop parameters) // 3. g_nvs_mutex (protects NVS read/write operations) // // VALID: acquire sensor_mutex, then control_mutex // INVALID: acquire control_mutex first, then sensor_mutex // -> this violates the ordering and CAN deadlock // Defensive coding: always use a finite timeout, never portMAX_DELAY // on mutexes between tasks. If the take times out, it is evidence // of a potential deadlock. Log it, escalate a fault. if (xSemaphoreTake(g_sensor_mutex, pdMS_TO_TICKS(100)) != pdTRUE) { ESP_LOGE(TAG, "Mutex timeout: possible deadlock detected"); fault_escalate(FAULT_MUTEX_TIMEOUT); return; }
```

7. Multi-MCU OTA — Why Atomicity Is Hard

Updating a single MCU is already complex — you need to download, verify, write, reboot, confirm, and handle every failure case. Updating three MCUs consistently is fundamentally harder, because now you can end up in a

partially updated state where some MCUs are on the new firmware and some are on the old.

Why does this matter? Because the firmware on different MCUs may be tightly coupled. The UART protocol between ESP32 and STM32 might change between firmware versions — new commands added, old ones deprecated, payload formats changed. If the ESP32 is running v2.1.0 and the STM32 is still on v2.0.5, they may be speaking incompatible protocol versions. The system could behave incorrectly or fail entirely.

This is why you need **version atomicity**: the system should only be considered successfully updated when ALL MCUs are running the new firmware. The IoT Job should only be marked SUCCEEDED at that point. If any MCU fails to update, all MCUs should roll back to the previous version.

The implementation challenge is coordination. The ESP32 updates itself first (it is the OTA coordinator). Then it sends firmware chunks to the STM32 over UART. Then it waits for confirmation from the STM32 that it has booted the new firmware. Only after all confirmations are received does it report SUCCEEDED. A 120-second deadline triggers a rollback if any MCU fails to confirm.

```
// The key insight: store the job ID in NVS before any MCU reboots. // After all MCUs confirm, load the job ID and report SUCCEEDED. // If anything times out, report FAILED and trigger rollback. typedef struct { char job_id[64]; bool esp32_confirmed; bool stm32_confirmed; uint32_t deadline_tick; // timeout = start + 120s } ota_commit_t; void ota_mcu_confirmed(mcu_id_t which) { if (which == MCU_ESP32) g_commit.esp32_confirmed = true; if (which == MCU_STM32) g_commit.stm32_confirmed = true; if (g_commit.esp32_confirmed && g_commit.stm32_confirmed) { // All MCUs confirmed -- safe to declare success job_report_succeeded(g_commit.job_id, COMBINED_FW_VERSION); shadow_update_fw_version(COMBINED_FW_VERSION); } else if (xTaskGetTickCount() > g_commit.deadline_tick) { // Timed out waiting for all confirmations -- rollback ESP_LOGE(TAG,"OTA commit timeout. esp32=%d stm32=%d", g_commit.esp32_confirmed, g_commit.stm32_confirmed); ota_rollback_all_mcus(); job_report_failed(g_commit.job_id, "partial_update_timeout"); } }
```

8. The 5-Level Fault Escalation Architecture

A closed-loop medical device needs a graduated response to faults. Not every fault warrants a full system shutdown — that would make the device unusable in practice. But some faults are so severe that the safest action is to immediately disable all actuators and wait for human intervention. A 5-level escalation architecture gives you this granularity.

Level 0 (Normal): Everything is working. All tasks are feeding the watchdog. All sensors are in range. Actuators are operating within limits. No action needed.

Level 1 (Warning): A non-critical anomaly was detected. A sensor reading is at the edge of its expected range but not clearly wrong. MQTT has been disconnected for more than 30 seconds. The appropriate response is to log the warning, publish it to the cloud shadow, and continue operating. No patient safety risk yet.

Level 2 (Soft Fault): A software subsystem has failed but can be recovered. A task missed its watchdog heartbeat once. A sensor is out of range for three consecutive readings. The appropriate response is to software-reset the faulting subsystem, log the fault to NVS, and notify the cloud. The device continues to operate with the recovered subsystem.

Level 3 (Hard Fault): The system cannot recover by resetting a subsystem. The hardware watchdog has fired, indicating the entire firmware hung. An unrecoverable sensor failure has occurred. The appropriate response is a full MCU reset, with the fault code stored in the RTC backup register (which survives reset) so it can be reported after reboot.

Level 4 (Safe State): Three Level 3 faults have occurred within 60 seconds — the system is in a loop of failures. This indicates a fundamental problem that software recovery cannot fix. The appropriate response is to immediately disable all actuators (the first action, before anything else), publish a CRITICAL_FAULT to the device shadow, and then block forever to let the hardware watchdog reset. The device will not return to operation until a cloud operator acknowledges the fault.

KEY INSIGHT: The order of operations in safe state matters critically: disable actuators FIRST, THEN log, THEN publish to cloud. If cloud publishing happens first and the network is unavailable (common during severe fault scenarios), the actuators would remain active while the device loops trying to connect. Always protect the patient first.

8. Lab Exercises

- [ ] Implement and unit test the CRC-16/CCITT-FALSE function against the 0x29B1 test vector.
- [ ] Build the RX state machine. Test 1000 frames with loopback. Verify 0 false-positives.
- [ ] Inject a 1-bit error at byte position 5 of a 10-byte payload. Verify NACK is sent.
- [ ] Demonstrate priority inversion with a binary semaphore. Measure the H-task latency spike.
- [ ] Fix with xSemaphoreCreateMutex(). Measure again. Calculate the improvement.
- [ ] Trigger a Level 4 safe state by causing 3 watchdog faults in 60 seconds.
- [ ] Verify actuators are disabled BEFORE the cloud publish in the safe state entry code.
- [ ] Implement the OTA atomicity guard. Test: kill STM32 UART mid-transfer. Verify rollback.

End of Tutorial 2. With this knowledge you can design and implement a production-grade 3-MCU firmware architecture with correct inter-MCU communication, safe FreeRTOS task design, and atomic multi-MCU OTA updates.